Dynamic Programming

Video - 133

Note :- This playlist is only for explanation of Qns & solutions.

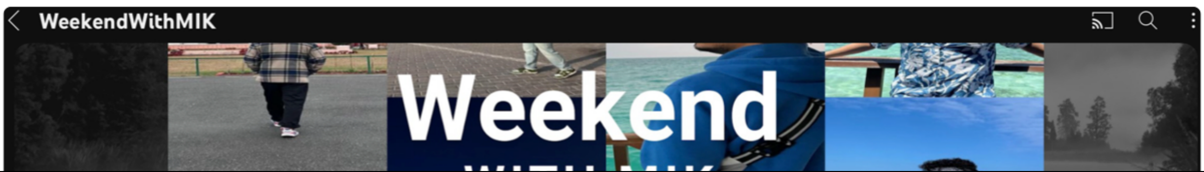
See my "DP Concepts & Qns" Paylist for understanding DP from scratch...

Leetcode - 3129
Medium

→ codestorywithmiK

→ CSwithMIK

→ codestorywithMIK



← My life behind scen
+
Tech News/updates

Motivation :-

Memoize the lessons from your past failures so you never have to process the same mistakes twice.
The optimal path to success is built simply by mastering one subproblem at a time.

MIK...

3129. Find All Possible Stable Binary Arrays I

Medium | Topics | Companies | Hint

You are given 3 positive integers `zero`, `one`, and `limit`.

A binary array `arr` is called **stable** if:

- The number of occurrences of 0 in `arr` is **exactly** `zero`.
- The number of occurrences of 1 in `arr` is **exactly** `one`.
- Each subarray of `arr` with a size greater than `limit` must contain **both** 0 and 1.

Return the *total* number of **stable** binary arrays.

Since the answer may be very large, return it **modulo** $10^9 + 7$.

Example 1:

Input: `zero = 1, one = 1, limit = 2`

Output: 2

Explanation:

The two possible stable binary arrays are `[1,0]` and `[0,1]`, as both arrays have a single 0 and a single 1, and no subarray has a length greater than 2.

Example 2:

Input: `zero = 1, one = 2, limit = 1`

Output: 1

Explanation:

The only possible stable binary array is `[1,0,1]`.

Note that the binary arrays `[1,1,0]` and `[0,1,1]` have subarrays of length 2 with identical elements, hence, they are not stable.

limit = 1

Example 3:

Input: `zero = 3, one = 3, limit = 2`

Output: 14

Explanation:

All the possible stable binary arrays are `[0,0,1,0,1,1]`, `[0,0,1,1,0,1]`, `[0,1,0,0,1,1]`, `[0,1,0,1,0,1]`, `[0,1,0,1,1,0]`, `[0,1,1,0,0,1]`, `[0,1,1,0,1,0]`, `[1,0,0,1,0,1]`, `[1,0,0,1,1,0]`, `[1,0,1,0,0,1]`, `[1,0,1,0,1,0]`, `[1,0,1,1,0,0]`, `[1,1,0,0,1,0]`, and `[1,1,0,1,0,0]`.

Constraints :-

1 <= zero, one, limit <= 200

Thought Process

zero → exactly count of 0s

one → exactly count of 1s

Each subarray having size > limit must contain both 0 and 1 limit = 2

1 0 0 0 1 1 1 0 1 1 0 0

3, 4, 5

limit = 2

1 0 0 0 1

"You cannot have more than Limit consecutive 0s/1s"

$\boxed{\text{zero} = 3}$, one = 2, limit = 2

0 1 0 1 1 ✓

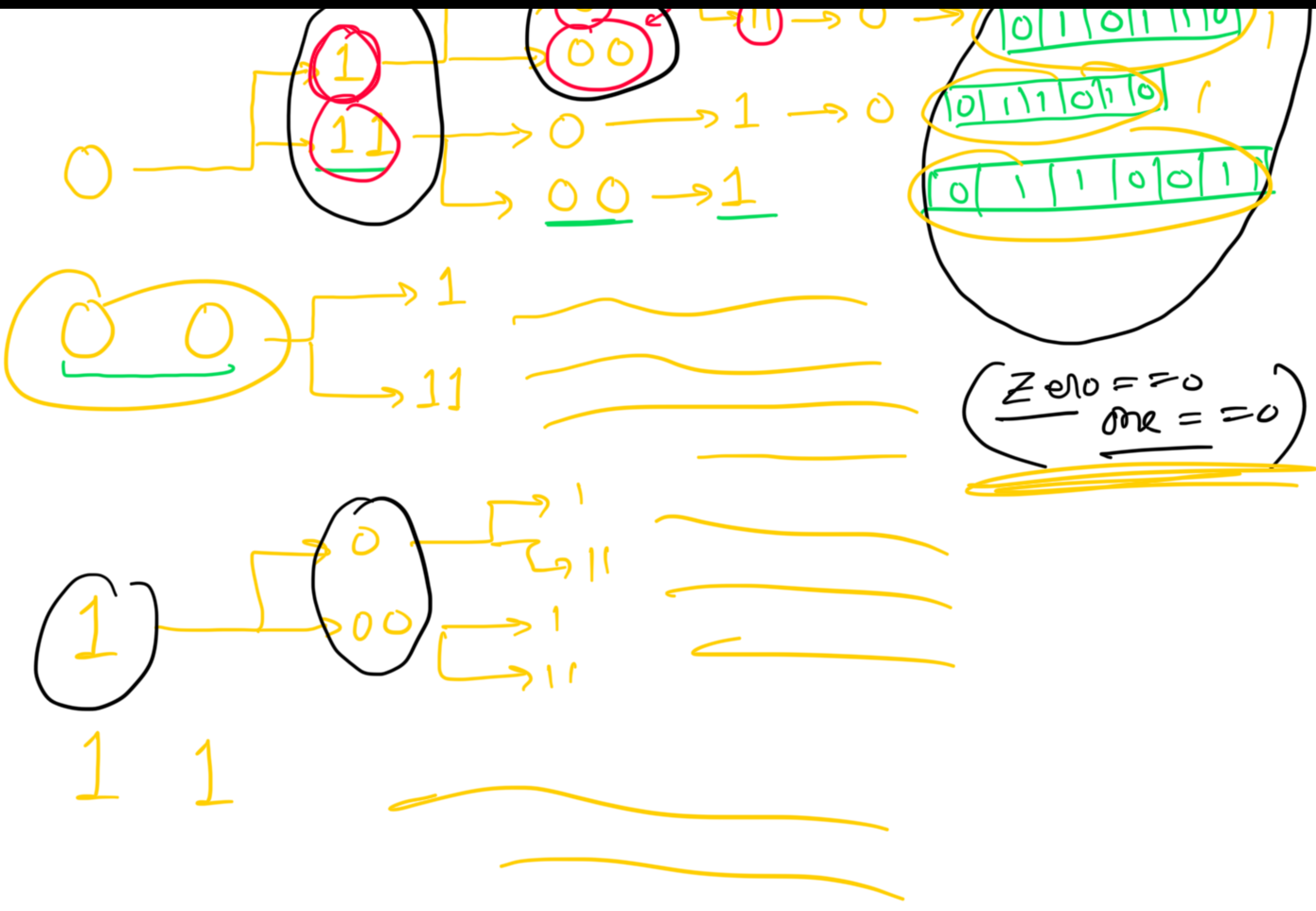
3
4 ✗
5 ✗

| 1 | 1 | 0 | 1 | 1 |

Did I explain the Problem statement to long ???

That was the only hard part of the problem. Now, see how easy it will be ☺

zero = 3 , one = 3 , limit = 2



Recursion.

limit = 2

Zero = 3

$\boxed{\min(\text{zero}, \text{limit})}$

min (one, limit)

Story To Code :-

Solve (onesLeft, zerosLeft, bool lastWasOne, limit){

```
if (onesleft == 0 && zerosleft == 0)
        return 1;

//explore 0s
if (lastWasOne == True) {
    for (int len = 1; len <= min(limit, zerosleft); len++)
    {
        result += Solve(onesleft,
                        zerosleft - len, False, limit)
    }
}

else {
    // Explore 1s
    for(int len = 1; len <= min(limit, onesleft); len++) {
        result += solve(onesleft - len, zerosleft,
                        True, limit);
```

}

}

return result;

}

startWithOne = Solve(one, zero, False, limit);

StartwithZero = Solve(one, zero, True, limit);

Bottom UP

(Using exact same code of Recursion).

```
int solve(int onesLeft, int zerosLeft, bool lastWasOne, int limit) {
    if(onesLeft == 0 && zerosLeft == 0) {
        return 1;
    }

    if(t[onesLeft][zerosLeft][lastWasOne] != -1) {
        return t[onesLeft][zerosLeft][lastWasOne];
    }

    int result = 0;
```

tru=1 / F=0

t[ ][ ][ ]

t[0][0][1] = 1

```
    if(lastWasOne == true) { //explore 0s
        for(int len = 1; len <= min(zerosLeft, limit); len++) {
            result = (result + solve(onesLeft, zerosLeft - len, false, limit)) % M;
        }
    } else { //explore 1s
        for(int len = 1; len <= min(onesLeft, limit); len++) {
            result = (result + solve(onesLeft - len, zerosLeft, true, limit)) % M;
        }
    }

    return t[onesLeft][zerosLeft][lastWasOne] = result;

}
```

t[0][0][0] = 1

t[ ][ ][0/1]

onesLeft zerosLeft lastWasOne

```
for(onesLeft = 0; onesLeft <= One; onesLeft++) {

    for(zeroLeft = 0; zeroLeft <= Zero; zeroLeft++) {

        if(onesLeft == 0 && zeroLeft == 0) {
            continue;
        }

lastWasOne = True;
        int result = 0;  // Exp 0s
        for(len = 1; len <= min(zeroLeft, limit); len++) {
            result = (res + t[onesLeft][zeroLeft-len][0]) % M;
        }
        t[onesLeft][zeroLeft][1] = result;


        result = 0;  // Exp 1s
```

```
lastWasOne = False {
   ↓
   0
        for( len = 1 ;  len <= min(onesleft , limit) ;  len++) {
            result = (res  +  f[onesleft-len][zeroleft][1]  ) % M ;
        }

        f[onesleft][zeroleft][0]  = result;
    }
}


a  =   f[one][zero][0] ;
b  =   f[one][zero][1] ;

return  (a+b) % M ;
```

[redacted]